ChatGPT

Advanced Study Guide: Language Model Objectives, Tokenization, and Evaluation Metrics

Introduction

Modern natural language processing models rely on **self-supervised pre-training objectives** and effective **tokenization schemes** to learn from massive text corpora. This study guide explores the key pre-training strategies (such as next-token prediction vs. masked language modeling and the "text-to-text" unified framework), popular subword tokenization methods (BPE, Unigram, and SentencePiece), the relationship between loss functions and perplexity, and the critical issue of data leakage in language model training and evaluation. We provide full mathematical derivations for loss and perplexity, explain how tokenization algorithms function, and discuss how to detect and mitigate dataset leakage. Reflection questions are included to reinforce understanding of these concepts.

Pre-training Objectives for Language Models

Pre-training a language model involves choosing an **objective function** that lets the model learn patterns from unlabeled text. Two widely used objectives are **autoregressive next-token prediction** and **masked language modeling (MLM)**. Both are self-supervised (no manual labels needed), but they differ in how the model sees context and predicts tokens:

- **Next-Token (Autoregressive) LM Objective:** The model learns to predict the next token in a sequence given all previous tokens. Formally, for a sequence of tokens $x_1, x_2, \dots, x_T$, the model is trained to maximize $P_\theta(x_t \mid x_{<t})$ for each position t. The training minimizes the negative log-likelihood (or cross-entropy loss) of the next token at each step:

$$\mathcal{L}_{\text{next-token}} = -\frac{1}{T}\sum\,.$$}^{T} \log P_\theta(x_t \mid x_{<t

This objective is used in **causal language models** like GPT-family: the Transformer's attention is typically masked so that each token only attends to earlier tokens (no future information). The model generates text one token at a time.

- **Masked Language Model (MLM) Objective:** The model learns to predict missing tokens in a sequence that has had some tokens masked or replaced. For example, given a sentence with certain words replaced by a special `[MASK]` token (or corrupted in some way), the model must predict the original words. In BERT's original MLM approach, a random sample (e.g. 15%) of input tokens are chosen; of those, most are replaced with `[MASK]` and some with random words. The model's transformer encoder processes the entire sequence (bidirectional context) and outputs a probability distribution over the vocabulary for each masked position. The loss is the cross-entropy of the true token at each masked position. If M positions are masked (or corrupted) in the input x, the MLM loss can be written as:

$$\mathcal{L}_{\text{MLM}} = -\frac{1}{M}\sum\,.$$}} \log P_\theta(x_i \mid x_{\text{all except } i

Unlike autoregressive LM, MLM uses *bidirectional context* (the model attends to both left and right context around the mask). Only the masked tokens contribute to the loss; unmasked tokens are just seen as context [1] [2]. This objective was introduced in BERT and related models, enabling the model to learn deep bidirectional representations of text.

Next-Token vs. Masked LM – Key Differences: Autoregressive models (next-token) are naturally suited for **text generation**, as they learn to continue sequences. They can assign a probability to any complete sequence by multiplying the probabilities of each next-token decision. **Perplexity** (described later) is well-defined for these models, serving as a metric of how well the model predicts test sequences. In contrast, masked LMs like BERT are not typically used to generate text in free-form; instead, they excel at **encoding** text for understanding tasks (classification, QA, etc.). A masked LM does not directly provide a probability for an entire sequence (since it doesn't generate it token by token), so perplexity is *not* straightforwardly applicable [3]. In fact, "perplexity" is usually **not reported for masked LMs** because their training objective doesn't naturally define a left-to-right sequence probability [3]. Instead, evaluation of MLMs is often via downstream tasks or masked word prediction accuracy.

Example: Consider the sentence "The cat sat on the [MASK]". A BERT-like MLM sees the entire sentence with "[MASK]" and might predict the masked token as "mat" with high probability if it learned that collocation. A GPT-like autoregressive LM would process "The cat sat on the" and then predict the next word, also hopefully assigning high probability to "mat." The difference is that GPT never saw the full sentence at once (it doesn't know about the future token), whereas BERT did but had the target hidden. This illustrates the bidirectional context advantage for MLMs (useful for understanding tasks), versus the unidirectional generation capacity of autoregressive models (useful for fluent text generation).

"Text-to-Text" Unified Pre-Training (T5)

A notable extension of language model pre-training is the **unified text-to-text framework** introduced by Raffel et al. in the T5 model (Text-To-Text Transfer Transformer). In this approach, *all tasks are formatted as text-to-text problems*: the model always takes text as input and produces text as output, regardless of whether the task is translation, summarization, question answering, or classification. By converting every problem into a text input and letting the model generate the text answer, **the same model and objective can be used across a wide variety of NLP tasks** [4].

During pre-training, T5 uses a variant of the masked language modeling objective called **span corruption** (or "fill-in-the-blank" denoising). Instead of masking individual tokens, T5 randomly selects spans of text and replaces each with a single mask token (like `<X>`) to indicate a gap. The model's task is to generate the missing span tokens as a sequence from that indicator. For example, the input might be "The cat sat <X> mat", and the target output for the model to generate would be "on the". This is done in a sequence-to-sequence setup: an **encoder** reads the corrupted text, and a **decoder** generates the missing text spans. This method combines aspects of MLM (corrupting the input) with an autoregressive generation on the output side.

The **text-to-text paradigm** offers a few advantages: - **Unified format:** Since everything is text-in/text-out, one can pre-train on a mixture of tasks or fine-tune easily to any task by appropriate prompt formatting. Raffel et al. (2020) emphasize converting all language problems into a text-to-text format [4]. - **Single**

model for all tasks: The same Transformer architecture with encoder-decoder can handle tasks that BERT-like (encoder-only) or GPT-like (decoder-only) models would separately handle. T5 achieved strong results across summarization, QA, classification, translation, etc., by *transfer learning* from this general text-to-text pre-training [5]. - **Pre-training on unlabeled data:** T5 was pre-trained on a massive cleaned dataset (C4: the "Colossal Clean Crawled Corpus" [6]) using the span-corruption objective, which is unsupervised. Then it was fine-tuned on specific tasks formatted as text-to-text.

Reflection: The text-to-text approach blurs the line between "language modeling" and "task-specific training" – essentially everything becomes a conditional text generation problem. A classification can be seen as generating a label word, translation as generating the translated sentence, etc. This required designing appropriate prompts for each task (e.g., prefixing inputs with a prompt like "translate English to German: ..." for a translation task in the input text). The unified model (T5) showed that this flexibility does not compromise performance; in fact, with sufficient scale and proper training, it *improved* transfer learning results across many benchmarks [5].

Subword Tokenization: BPE, Unigram, and SentencePiece

Raw text is typically composed of characters or whitespace-separated words, but using characters as the basic unit can make sequence lengths very long, and using whole words fails for rare or unknown words (out-of-vocabulary issues). **Subword tokenization** offers a compromise: break words into smaller pieces (subwords) such that common word parts are retained as single tokens, but rare words can be built from multiple subword tokens. Subword units allow an **open vocabulary** – the model can represent any word (even unseen ones) by a sequence of subword tokens. We discuss three popular subword tokenization schemes: **Byte Pair Encoding (BPE)**, **Unigram Language Model (Unigram LM)** tokenization, and **SentencePiece** (a toolkit which can implement BPE/Unigram and handles text seamlessly).

Byte Pair Encoding (BPE)

BPE is an algorithm originally from data compression (Gage, 1994) that was adapted by Sennrich et al. (2016) for word segmentation in NLP [7]. The key idea is to start with individual characters as tokens and then iteratively **merge the most frequent adjacent token pair** into a new token. Over many merges, frequent sequences of characters (including multi-character sequences or whole words) become single tokens. This produces a fixed-size vocabulary of subwords that optimally balances between single characters and full words.

- **Initialization:** Begin with a "vocabulary" containing every character (and a special end-of-word symbol, often `</w>` or whitespace marker) as tokens [8]. So initially, each word in the corpus is represented as a sequence of character tokens. For example, the word "lower" might start as `l o w e r </w>` (where `</w>` marks end-of-word) [8].

- **Iterative Merging:** Count all pairs of consecutive tokens in the training corpus. Identify the pair with the highest frequency (occurring most often together). For instance, if "l" and "o" frequently occur as "lo", that pair might be the most frequent. Merge that pair into a new single token (e.g., "lo"). This reduces the total number of tokens in the corpus (since every occurrence of that pair is now one token instead of two) and adds a new entry to the vocabulary for the merged token [9] [10].

- **Repeat:** After each merge, update the token sequence (replacing all occurrences of the pair with the merged token) and recompute frequencies. Continue merging the most frequent pair until you have performed a pre-specified number of merges, or reached a desired vocabulary size. Each merge increases vocabulary size by 1 (since a new token is introduced) [11].

- **Result:** A vocabulary that includes all original characters plus all merged multi-character tokens formed. Common substrings become their own tokens. Rare words that were never seen in full remain split into smaller pieces, but no truly unseen string of characters is impossible to represent (since at worst you fall back to characters). The vocabulary size (which equals initial char set + number of merges) is a hyperparameter typically chosen around 30k–50k for many models [12] [10]. BPE *does not require* an explicit list of words (no fixed word vocabulary limit); it *creates* subword units that allow open-vocabulary modeling [7].

One great benefit of BPE is that it **effectively addresses the "rare word" problem** in neural machine translation (the context Sennrich et al. introduced it). By encoding rare words as sequences of subword units, the model can learn to translate or understand pieces of the word (like morphemes) and thereby handle words it never saw as a whole during training [12] [13]. Sennrich et al. showed that a NMT model using BPE subwords can even generate completely novel words by combining subwords, and improves translation of infrequent terms compared to a baseline with limited vocabulary [13].

> **BPE example:** Suppose our corpus contains words "low", "lower", "newer", "wider". Initially, tokens are individual letters plus `</w>`. Likely merges would happen like: - Most frequent pair might be "l+o" in "lo", or "e+r" in "er", etc. After a few merges, we might get tokens like "low", "er" etc. - Eventually, "lower" might be tokenized as "low" + "er</w>", "newer" as "new" + "er</w>", "wider" as "wid" + "er</w>", "low" as "low</w>". So subwords like "low", "er" become part of vocab. This is just illustrative – actual merges depend on frequencies. The key is common substrings like "low" and "er" were merged due to high frequency across words.

Formally, BPE is greedy and **deterministic** given the merge list: once you learn the merges, any new word is tokenized by greedily applying the longest matching merges (usually implemented to go from left to right, always merging the largest possible token that is in the vocabulary). This means the tokenization result is unique. The algorithm can be summarized as a simple loop and is easy to implement; indeed Sennrich's paper even provided a minimal Python implementation [14] [15]. BPE's simplicity and effectiveness led to its adoption in many large language models (GPT-2, GPT-3's tokenizer is based on BPE, etc., though often with some modifications like byte-level BPE in GPT-2 for unicode handling). Sennrich et al. noted: *"We adapt byte pair encoding (BPE), a compression algorithm, to the task of word segmentation. BPE allows for the representation of an open vocabulary through a fixed-size vocabulary of variable-length character sequences, making it a very suitable word segmentation strategy for neural network models."* [7]

Unigram Language Model Tokenization

The **Unigram LM** tokenization approach, introduced by Kudo (2018), takes a different strategy: rather than merging pairs, it starts with a large pool of candidate subword pieces and *removes* the ones that are not useful, until the vocabulary shrinks to the desired size. It's based on a **unigram language model** over subwords: essentially a probabilistic model that assumes each token in a segmented sequence is generated independently (a "unigram" model, ignoring context). The segmentation of a word into tokens has a

probability (the product of the tokens' probabilities in the model). The goal is to choose a vocabulary that maximizes the likelihood of the training corpus according to this model.

How Unigram Tokenization works: - Start with a very large initial vocabulary of subword pieces. This could be all substrings up to a certain length that appear in the corpus (which would be huge), or a heuristic selection. In practice, Kudo initializes with the results of something like byte-level BPE with a high merge count, to get an oversize vocab containing many possible subwords. - Assign an initial probability to each subword piece. Then iteratively perform an Expectation-Maximization (EM) procedure: calculate the probability of each possible segmentation of the training text under the current model (E-step), and update the token probability distribution to maximize likelihood (M-step). Rather than explicitly doing heavy EM on a huge vocabulary, a faster approximation is used: **prune** tokens that only contribute a tiny amount to overall likelihood. Specifically, they remove the tokens whose absence would have minimal impact on the corpus likelihood, until the vocabulary size is down to the target (e.g., 32k). - The result is a vocabulary optimized such that any text can be segmented (likely in many possible ways) and the chosen vocabulary is the one that best fits the training data distribution in a likelihood sense.

Notably, in a **unigram model**, there may be multiple valid segmentations of a word (each segmentation is a sequence of tokens that concatenates to the word). The model can assign probabilities to each segmentation. At tokenization time, one typically picks the *maximum likelihood segmentation* for each word given the model's token probabilities (this can be done efficiently with Viterbi algorithm). This means the segmentation is not necessarily unique in theory, but we choose the best one for consistency (or sometimes sample one randomly for data augmentation, as discussed next).

An advantage of Unigram LM tokenization is that it can more naturally support **subword regularization** – the idea of sampling different segmentations each time a sentence is seen during training to make the model robust to tokenization variations. Kudo & Richardson (2018) describe that **SentencePiece implements both BPE and Unigram** and supports such random sampling of segmentations for augmentation [16]. For example, if a word "internationalization" can be segmented as "international + ization" or "internation + al + ization" with near-equal likelihood, the model could randomly pick one in one epoch and another in a later epoch, so the model isn't tied to one arbitrary segmentation.

In summary, the Unigram approach is a **top-down** pruning method vs. BPE's **bottom-up** merging. Both aim to find frequent subwords, but Unigram does so by probability and removal, which often yields similar vocabularies to BPE. Empirically, Kudo (2018) found Unigram LM performs on par with BPE for machine translation, with some advantages when used with subword regularization. In fact, a study noted that *"BPE and unigram language models show almost comparable performance"* in many cases [17], so the choice may come down to implementation and preferences (BPE's deterministic nature vs. Unigram's probabilistic model).

SentencePiece

SentencePiece (Kudo & Richardson, 2018) is not a new tokenization algorithm per se but a **framework/ toolkit** that made using subword models easier and language-independent. The problem it solved is that many prior tokenization tools expected pre-segmented input (e.g., already split on whitespace/punctuation) and were language-specific. SentencePiece treats the input as a raw stream of characters (it even considers whitespace as just another symbol) and learns subword units directly from that. This allows truly end-to-end

data processing: you don't need a separate pre-tokenization step for splitting on spaces or handling languages like Japanese that don't use spaces [18].

Key features of SentencePiece: - It introduces a special symbol (often ▁ underscore) to represent whitespace explicitly. For example, the text "Hello world" internally becomes "▁Hello▁world" as a sequence of characters including the underscore for spaces. This way, after training, a generated sequence of subword tokens can include these underscores to indicate where spaces should be when rendering text [18]. This avoids ambiguous detokenization. - SentencePiece can train a BPE model or a Unigram model (it implements both algorithms) on the raw text [16]. The user specifies the desired vocabulary size and algorithm. The output is a vocabulary file and a model that can tokenize new text in the same way the training was done. - It's language independent: no language-specific rules or segmentation needed. This is crucial for languages without whitespace word boundaries (Chinese, Japanese, etc.) because older tools often required a separate morphological segmenter for those languages. With SentencePiece, you just feed in the raw text in any language. - It also provides an efficient C++ implementation and easy integration with neural network frameworks (there are Python bindings, etc.), making it convenient for NLP pipelines.

Kudo & Richardson highlight that existing tools assumed pre-tokenized input (split into words), but *"SentencePiece can train subword models directly from raw sentences, which allows us to make a purely end-to-end and language independent system."* [18]. This has become the standard in many projects; for instance, Google's translation and language models use SentencePiece to generate their subword vocabularies. Even the popular HuggingFace Tokenizers library has compatibility with SentencePiece models for various multilingual BERT and T5 models.

Example: If using SentencePiece BPE with vocab size 10k on English text, it will learn subwords similarly to traditional BPE, but it will include the underscore symbol. The phrase "New York" might end up tokenized as "▁New" + "▁York" (if it learns those as separate tokens including the space), or "▁New" + " York" (less likely since the space is a separate token in SentencePiece's paradigm). In practice, SentencePiece ensures that every token except those at start of text includes the leading underscore if it originally followed a space.

In practice, SentencePiece is often mentioned alongside BPE/Unigram because it *is the tool* that implements them. For example, the tokenizers for models like GPT-2/GPT-3 used a byte-level BPE (OpenAI's implementation), while many others like BERT (WordPiece algorithm), RoBERTa (BPE), and T5 (Unigram via SentencePiece) rely on similar subword ideas. **WordPiece**, another subword algorithm (used in the original BERT), is conceptually similar to BPE in that it iteratively builds a vocab by splitting words, but it uses a different selection criterion (based on maximizing likelihood of the training data similar to Unigram, but greedily adding one token at a time). WordPiece is essentially an earlier approach from Schuster and Nakajima (2012) and was deployed in Google's production systems; it hasn't been explicitly covered here since BPE and Unigram are more commonly referenced in recent literature, but it's good to know it as a variant.

Takeaway: Subword tokenization is essential for large language models. It lets models handle any input text with a finite vocabulary, mitigating out-of-vocabulary issues and reducing the vocabulary size needed (which keeps model embedding sizes manageable). BPE and Unigram are two effective strategies for generating subwords. **Sennrich et al. (2016)** demonstrated the power of BPE in neural MT by enabling open-vocabulary translation with a fixed vocab of subwords [7]. **Kudo & Richardson (2018)** provided the SentencePiece toolkit implementing these methods, allowing direct training on raw text and language-

independent tokenization [18]. Modern LMs often use one of these methods; for example, GPT-3's tokenizer is a variant of byte-level BPE, and T5's tokenizer is a SentencePiece Unigram model (with a 32k vocab from the C4 corpus).

Language Model Loss and Perplexity

When training language models (especially autoregressive ones), **cross-entropy loss** is the standard objective, and **perplexity** is a standard evaluation metric. Understanding the math behind these is important, as well as how they relate to each other and to model performance.

Cross-Entropy Loss for Language Modeling

The cross-entropy loss measures the difference between the model's predicted probability distribution over tokens and the true distribution (which in the case of a next-word prediction is a one-hot vector for the correct next token). For a single position t in a sequence, if the true next token is w_t, the cross-entropy (negative log-likelihood) loss is $-\log P_\theta(w_t \mid \text{context})$. For a whole sequence $X = (x_1, x_2, \ldots, x_T)$, the average loss per token is:

$$ \mathcal{L}*{\text{LM}}(X) = -\frac{1}{T} \sum\,, $$}^{T} \log P_\theta(x_t \mid x_1,\ldots,x_{t-1

assuming a next-token modeling scenario. This is exactly the objective we described earlier for autoregressive models. In practice, models often include an end-of-sequence token and the sum may go up to T including that, but the details vary. The key is that minimizing this loss means the model is assigning higher probability to the ground-truth token at each step.

For masked language models, a similar loss is computed but only over the masked positions (as shown above in MLM objective). Each masked position's loss is $-\log P_\theta(\text{correct token}|\text{mask context})$, and averaged over masks.

Cross-entropy loss can also be expressed in terms of the model's predicted probability distribution q and the true distribution p (one-hot). If p_i is 1 for the correct token and 0 for others, and q_i is model probability for token i, then $\text{CrossEntropy}(p,q) = -\sum_i p_i \log q_i = -\log q_{\text{correct}}$. This makes it clear that lowering cross-entropy is equivalent to increasing the probability assigned to the correct token.

Perplexity and Its Derivation

Perplexity (PPL) is a measurement derived from the cross-entropy that is often more intuitive in language modeling. The perplexity of a model on a sequence (or a corpus) is defined as the **exponential of the average negative log-likelihood**. If L is the average cross-entropy loss (in natural log), then:

$$ \text{Perplexity} = \exp(L) = \exp!\Big(-\frac{1}{T}\sum_{t=1}^{T} \log P_\theta(x_t \mid x_{<t})\Big)\,. $$

Equivalently, $\text{PPL}(X) = \big(\prod_{t=1}^T P_\theta(x_t \mid x_{<t})\big)^{-1/T}$, which is the reciprocal of the geometric mean of the sequence probability. In simpler terms, perplexity is the **effective average branching factor** of the model: how many choices the model is "as confused between" at each step, on

average [19]. For example, if a model has a perplexity of 50 on some dataset, it means on average it's about as uncertain as if it had to choose uniformly among 50 possibilities for the next token at each step.

Perplexity provides an interpretable scale (the lower, the better): - A perfect model that assigns probability 1 to the correct next token (and 0 to others) would have cross-entropy 0 and thus perplexity 1 (no confusion, one choice). - A model that is equivalent to randomly picking among V equally likely tokens for each next word (where V is vocabulary size) would have cross-entropy $\log V$ and perplexity V. - Real models fall in between these extremes.

Because $\text{PPL} = \exp(\text{Cross-Entropy})$, minimizing cross-entropy is equivalent to minimizing perplexity. Authors often report perplexity as it's more directly understandable. Also, note if log base-2 is used for cross-entropy (common in some information theoretic contexts), then perplexity is 2^{H}, but if natural logs are used, it's e^{H} [20]. The base doesn't matter fundamentally for comparisons, just convention.

Relation between Loss and PPL: If a model's average negative log-likelihood on a test set is $L = 2.3$ (nats, which is about $\log(10)$), then perplexity is $e^{2.3} \approx 10$. If you improve the model so that the loss is 2.0, perplexity becomes $e^{2.0}\approx 7.39$. Thus a lower cross-entropy corresponds to a lower perplexity, indicating the model is less "perplexed" by the text. It's common in research papers to say something like "Model A achieved a validation perplexity of 20, whereas Model B's perplexity was 25, so Model A is better."

One **important caveat**: perplexity values are only comparable when computed on the same dataset *and using the same tokenization*. If one model is trained with a different tokenizer (say word-level vs subword-level), their perplexities are not directly comparable because the sequence of prediction events is different [21]. For example, a character-level model might have many more prediction steps for a sentence than a word-level model, which will affect perplexity. So always compare like to like.

Perplexity and model types: As noted, perplexity is well-defined for **autoregressive language models** (which produce a probability for an entire sequence by sequential factorization). For **masked LMs**, one could try to compute a pseudo-perplexity by masking each word in turn and measuring its probability (this has been attempted to score sentences with BERT), but it's not a standard or straightforward metric. The Hugging Face documentation also notes that perplexity "applies specifically to classical (causal) language models and is not well defined for masked language models like BERT" [3].

Mathematical Derivation: Starting from cross-entropy H (per token): $$ H = -\frac{1}{T}\sum_{t=1}^T \log P(x_t|x_{<t})\,. $$ Exponentiating (with base e) gives: $$ \exp(H) = \exp!\Big(-\frac{1}{T}\sum_{t=1}^T \log P(x_t|x_{<t})\Big) = \Big(\prod_{t=1}^T P(x_t|x_{<t})\Big)^{-1/T}\,, $$ which is exactly the perplexity definition above. If we had used log base 2, $\text{PPL} = 2^H$. The notion of perplexity comes from information theory: if you view H (in bits) as the entropy per word, then 2^H is the effective number of choices for a word. Similarly, e^H does the same using natural logs. Thus perplexity is essentially measuring the **surprise** of the model for the true data (lower surprise = better model).

Why Perplexity? In language modeling, perplexity has been a primary intrinsic metric because it correlates with how well a model predicts language. A lower perplexity generally means better predictive performance, which often translates to better downstream task performance as well (though not always perfectly). For example, GPT-2 was partly notable for achieving very low perplexities on test corpora,

corresponding to its fluent generation ability. It's worth noting, however, that extremely low perplexity might also indicate **overfitting** or **data leakage** if the model saw the test data during training (we'll cover leakage next). In a healthy scenario, perplexity dropping on a held-out set indicates the model generalizes better.

In summary, **cross-entropy loss** is minimized during training, and **perplexity** is just an exponentiated form of that loss used for evaluation. They are tightly linked: *"Perplexity is defined as the exponentiated average negative log-likelihood of a sequence. This is also equivalent to the exponentiation of the cross-entropy between the data and model predictions."* [19] [20].

As a concrete example: imagine a trivial model that always predicts each next word with probability 0.1 for the correct word and spreads 0.9 over others (so basically 10% chance to be right). Its per-token cross-entropy loss (with natural logs) would be $- \log(0.1) = 2.3026$ nats, giving a perplexity of $e^{2.3026} = 10$. If another model manages 20% next-word accuracy on average (0.2 probability for correct token), loss $-\log(0.2)=1.6094$, perplexity $e^{1.6094}=5$. So the second model is much better (perplexity 5 vs 10).

Data Leakage and Contamination in Training

Data leakage refers to any situation where information that should be separated between training and evaluation finds its way across the divide, leading to overly optimistic performance estimates or unintended memorization. In the context of language model pre-training and evaluation, a critical form is **dataset contamination**: when the evaluation (validation/test) data is present (partially or wholly) in the training data. This can happen especially in large-scale web crawled corpora – for example, if a model is trained on a dump of the internet, it might inadvertently include articles or benchmarks that later are used to test it. If not handled, the model might simply memorize those and ace the test not by generalization but by recall.

Why it's a problem: If a benchmark's test set is leaked into training, the model's perplexity on that test or its accuracy on tasks will be misleadingly high because it has seen the answers. As one work describes, *"Dataset contamination, where evaluation datasets overlap with pre-training corpora, inflates performance metrics and undermines the reliability of model evaluations."* [22] In other words, we might think our model is great, but it's partly because it already knew the test.

Forms of leakage: - **Exact overlap:** e.g., the exact same sentence or document appears in training and test. Models can memorize long passages verbatim, especially if they are repeated or distinctive. High overlap can often be detected by n-gram matching or hashing techniques. - **Partial overlap:** e.g., a test paragraph is almost the same as a training one except minor changes, or the answer to a test question appeared somewhere in training text. This is harder to detect but still problematic. - **Leakage via preprocessing:** e.g., using some knowledge that was derived from test data in training. (In ML, a classic example is normalizing features using the whole dataset mean – leaking test info into train.) - **Intentional or unintentional insertion:** In some cases, test data may have been in a popular dataset (like Wikipedia) which the model was pre-trained on. This was noted with GPT-3: many NLP benchmarks are on the internet, and GPT-3's vast training set (Common Crawl etc.) likely contained those test instances ("contamination"). The authors of GPT-3 did attempts to measure and mitigate this.

Leakage Detection: How can we tell if our model or training data is compromised by leakage? 1. **Data Deduplication:** A straightforward approach is to **deduplicate** the training corpus against the test sets. For example, for each test sentence or passage (or substantial n-gram from it), search the training data. The

Pile dataset (Gao et al., 2020) creators explicitly deduplicated their corpus [23] to reduce redundancy. OpenAI also mentions removing data overlaps. If any near-duplicates are found, they can be removed or the test replaced. 2. **Perplexity-based signals:** If the model assigns an *extremely high probability (low perplexity)* to a long sequence (much higher than to comparable sequences), that could indicate memorization. A model typically shouldn't be almost certain of a long, unique sequence unless it saw it before. Research has used this idea to find memorized training data in LMs. For instance, one can prompt the model and see if it spills out a block of text verbatim with high confidence. In fact, one study explicitly looked for **low-perplexity sequences** in an LLM's outputs as a way to detect memorized training data [24]. They define "memorization" as cases where the model outputs content that can be traced back to a unique source in training, often containing specific knowledge or private data [24]. If a sequence of output has perplexity far lower than typical language (or the model's overall perplexity), it might be something the model is reciting from memory (thus a leakage of training data content). 3. **Embedding-based detection (KDS):** A recent advanced method is the **Kernel Divergence Score (KDS)** [25]. This method (Choi et al., 2025) proposes fine-tuning a model on the suspected benchmark and measuring how the model's internal representations shift for data it has seen vs not seen. The intuition is: fine-tuning will significantly change the model's representation for examples it had not seen before (as it learns them), but if an example was already seen (and memorized), fine-tuning on it won't change the model much. KDS quantifies this by comparing representation similarity matrices before vs after fine-tuning [25]. A high score indicates likely contamination. They report near-perfect correlation of KDS with actual contamination levels in controlled tests [26]. This is a sophisticated approach when direct data comparison isn't feasible (like if data is too large or privacy concerns). 4. **Knowledge probing and QA:** Sometimes you can catch leakage by asking the model something it shouldn't know unless it saw specific data. For example, if a model recites an answer that matches some hidden test or private record verbatim, that's a sign it memorized it.

Prevention and Mitigation: The best cure is prevention – carefully curate training data to exclude anything that overlaps with evaluation. For benchmarks, ensure they are genuinely "held-out." For instance, if you're training on a huge scrape, remove articles that are in your evaluation dataset (which might be known by title or content). The Pile authors performed an "in-depth exploratory analysis" of their data, documenting potentially concerning aspects [27] – presumably including overlaps or sensitive info – to inform users. OpenAI when training Codex (for code) filtered out any code from evaluation sets of programming challenges. Deduplication of training data in general can help not just for leakage but also for reducing overfitting; *Lee et al. (2021)* found that deduplicating a large corpus can reduce a language model's tendency to memorize and even improve perplexity slightly [28].

Leakage vs. Generalization: It's worth noting that large LMs can generalize, so seeing related information isn't always a leak in the harmful sense. The boundary can blur: if a model saw millions of similar QA pairs, is it "leakage" that it does well on a new QA pair? Not really – that's just generalization from training distribution. Leakage is usually reserved for *exact* (or nearly exact) overlap or the model exploiting unintended information. An example outside language modeling: if in a medical dataset, patient IDs accidentally appear in both train and test with labels, the model could just memorize the ID->label mapping (that's leakage and it shortcuts real learning).

In language models, one famous demonstration of memorization leakage was by Carlini et al. (2021), who showed that GPT-2 could sometimes regurgitate entire training passages (including personal info) when prompted cleverly. They define a sequence as *memorized* if the model can generate it with very low entropy (high probability) and it's a unique training sequence. They indeed found examples of addresses, phone numbers, etc., in GPT-2's generations – a privacy concern [24].

From an evaluation standpoint, if your test perplexity looks **too good to be true**, consider checking for contamination. For example, if a model scores perplexity 5 on WikiText (a standard test set) whereas previous models had 20, one should suspect it might have seen WikiText during training.

In summary, **data leakage detection** involves a mix of data analysis and model behavior analysis: - Direct data analysis (overlap checks, deduplication) is a first line of defense. - Analyzing model outputs or internal stats (like distribution of perplexities) can reveal suspect cases of memorization. - New research tools like KDS can quantify contamination even when direct overlap isn't obvious, by how the model's knowledge changes when fine-tuned [25].

It's crucial for ensuring **fair evaluation**. If not addressed, we risk attributing a model's performance to general intelligence or training technique improvements, when in fact it might just be because it leaked the test data. As one study puts it, measuring contamination is essential to ensure evaluations "genuinely reflect a model's ability to generalize to unseen data, rather than relying on memorized examples" [22].

Having covered objectives, tokenization, loss metrics, and data leakage, we have a comprehensive view of the fundamentals of training and evaluating language models. Next, use the reflection questions to test and solidify your understanding. Each question is designed to make you think critically about the material; you can attempt to answer them and even use an AI assistant to discuss or check your reasoning.

Reflection Questions

1. **Next-Token vs. Masked LM:** Explain in your own words why a masked language model like BERT cannot be straightforwardly evaluated with perplexity. What makes the perplexity calculation valid for autoregressive models but problematic for MLMs?

2. **Text-to-Text Training:** What is the advantage of the "text-to-text" unified approach (as used in T5) compared to having separate model architectures for tasks like classification or translation? Can you think of any drawbacks or challenges of the text-to-text approach?

3. **BPE vs. Unigram Tokenization:** Both BPE and Unigram LM end up with a vocabulary of subword units. How do their algorithms differ in constructing that vocabulary? Consider how each would handle a new word it's never seen – would the tokenization outcome differ between BPE and Unigram?

4. **Tokenization and Vocabulary Size:** Why is it important to have a fixed vocabulary size for subword tokenization in neural networks? What might happen if the vocabulary is too large or too small? (Think in terms of embedding matrix size, model memory, as well as too aggressive splitting.)

5. **Loss and Perplexity Calculation:** Suppose a very tiny language model assigns probabilities to a test sequence of 3 tokens as 0.5, 0.1, and 0.1 for the correct tokens respectively. Compute the average negative log-likelihood (cross-entropy) and perplexity for this sequence. (Use base-e logs for calculation.)

6. **Interpreting Perplexity:** Model A has a perplexity of 30 on a certain dataset, and Model B has a perplexity of 50. What does that tell you about their relative performance? Is it possible that Model B could still perform better on a downstream task (like sentiment analysis) even though its perplexity is higher? Why or why not?

7. **Data Leakage Scenarios:** Give an example (hypothetical or real) of how test data could accidentally end up in a pre-training corpus. If you discovered after training that some test items were in your training data, what steps could you take to quantify and address the issue?

8. **Leakage vs. Memorization:** Large language models sometimes memorize training facts (like a specific quote or personal detail). Why is memorization of training data considered a form of "leakage" or problem in language models? In what cases might memorization be acceptable or even desired?

9. **Detection Methods:** How might you use the model itself to detect if it has potentially memorized a piece of text? (Hint: think about prompting the model or analyzing likelihoods.) What are the limitations of this approach?

10. **Tokenization Effects on Loss:** Two models are identical in architecture and training procedure, but one uses a character-level tokenizer and the other uses a BPE tokenizer with 30k vocab. They are evaluated on the same test text. How would their reported perplexities differ, and why is this not a completely fair comparison of their modeling power?

Feel free to explore these questions with pen and paper, or discuss them with peers or an AI. Being able to articulate these answers is a strong sign of mastery over objectives, tokenization, loss metrics, and data leakage concerns in modern NLP model training [7] [18] [19] [22]. Good luck!

---

[1] [2] web.stanford.edu
https://web.stanford.edu/~jurafsky/slp3/11.pdf

[3] [19] [20] [21] Perplexity of fixed-length models
https://huggingface.co/docs/transformers/en/perplexity

[4] [5] [6] [1910.10683] Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer
https://arxiv.org/abs/1910.10683

[7] [8] [9] [10] [11] [12] [13] [14] [15] Neural Machine Translation of Rare Words with Subword Units
https://aclanthology.org/P16-1162.pdf

[16] [17] [18] SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing.
https://aclanthology.org/D18-2012.pdf

[22] [25] [26] ICML Poster How Contaminated Is Your Benchmark? Measuring Dataset Leakage in Large Language Models with Kernel Divergence
https://icml.cc/virtual/2025/poster/43619

[23] How Much is Enough? The Diminishing Returns of Tokenization Training Data
https://arxiv.org/html/2502.20273v1

[24] Low-Perplexity LLM-Generated Sequences and Where To Find Them
https://arxiv.org/html/2507.01844v1

[27] [2101.00027] The Pile: An 800GB Dataset of Diverse Text for Language Modeling
https://arxiv.org/abs/2101.00027

[28] [PDF] Measuring memorization in language models via probabilistic ...
https://afedercooper.info/paper/hayes2024npextraction.pdf